Q1. Timers

Consider the LED flashing program in Listing 1. Given a CPU frequency of $8\,\mathrm{MHz}$ answer the following.

Listing 1: Source listing for Timer Question.

```
1  #include "pio.h"
2
3  void tempo(uint16_t bravo) {
4      while (TCNT1 < bravo)
5          continue;
6      TCNT1 = 0;
7  }
8
9  void flasher(uint16_t huey, uint16_t dewey, uint16_t louie)
10 {
11     TCCR1B = 0x05;
12     TCNT1 = 0;
13
14     while (louie--)    {
15         pio_output_high(LED1);
16         tempo(huey);
17         pio_output_low(LED1);
18         tempo(dewey - louie);
19     }
20 }
21
22 int main (void)
23 {
24     system_init();
25     pio_config_set(LED1, PIO_OUTPUT_LOW);
26     flasher(10, 400, 10);
27
28     return 0;
29 }
```

(a) What is the value of the prescaler?

(b) What is the resolution of the timer?

(c) What is the maximum period (delay) of the timer?

(d) How many times will the LED flash?

(e) Magic numbers are used in Listing 1. Add appropriate lines of code and rewrite others to show how magic numbers are removed from this program.

(f) Neglecting the time it takes to call `pio_output_low` and `pio_output_high`, determine the duty cycle of the LED. Assume the LED is active high.

Q2. Concurrency

The main routine for a periodic task scheduler is shown in Listing 2.

Listing 2: Periodic task scheduler

```
1
2  /* Periodic task scheduler */
3  int main (void)
4  {
5      init_game_parameters();
6
7      task_t tasks[NUM_TASKS] =
8      {
9      {.func = get_button, .period = TASK_RATE / BUTTON_RATE},
10
11     {.func = make_display, .period = TASK_RATE / DISPLAY_RATE},
12     {.func = update_game, . period = TASK_RATE / GAME_RATE}
13     };
14
15     task_schedule(tasks, NUM_TASKS);
16     return 0;
17 }
```

Complete Listing 3 to provide a paced loop version of the periodic task scheduler given in Listing 2. Use the same task names and appropriate pacer function calls to achieve similar functionality.

Listing 3: Paced loop task scheduler

```
1  /* Paced loop scheduler */
2  int main (void)
3      {
4
5
6      pacer_init(GAME_RATE);
7
8
9
10
11
12
13
14
15
16
17
18
19     return 0;
20 }
```